📦 DESARROLLO EN ENTORNO CLIENTE

Objetos Literales en JavaScript

Guía definitiva de sintaxis literal, encapsulación de propiedades y métodos con `this`, estructuras complejas anidadas, privacidad y serialización con formato JSON.

🏷️ Sintaxis y Notación

Creación directa con llaves `{}`, notación de punto vs corchetes, adición dinámica y borrado con el operador `delete`.

🪪 Métodos y 'this'

Inclusión de funciones como miembros del objeto y uso de `this` para manipular el estado interno con total autonomía.

🎓 Estructuras Anidadas

Composición de objetos dentro de objetos y arrays internos, con métodos que computan medias y estadísticas.

🔤 Arrays de Objetos y JSON

Colecciones para catálogos y tablas, inspección de claves con `Object.keys()` e intercambio con `JSON.stringify()` y `JSON.parse()`.

Profesor Ramón Abramo
Especialista en Programación Web Frontend · Más de 25 años de experiencia docente

Edición Académica 2026
Santander · España

📑 **ÍNDICE GENERAL DE CONTENIDOS** (Haz clic en cualquier sección para navegar)

📦 Portada Oficial y Presentación	Pág. 1	1. ¿Qué es un objeto literal y cuál es su sintaxis?	Pág. 2
2. Miembros públicos y privacidad en notación literal	Pág. 3	3. Ejemplo 1: Manipulación de Objeto Producto	Pág. 4
4. Ejemplo 2: Métodos y palabra reservada 'this'	Pág. 5	5. Ejemplo 3: Objetos anidados y arrays internos	Pág. 6
6. Ejemplo 4: Catálogos con Arrays de Objetos	Pág. 6	7. Ejemplo 5: Object.keys() y Serialización JSON	Pág. 7
8. Top 5 Errores Frecuentes y Soluciones	Pág. 7		

🏷️ 1. ¿Qué es un objeto literal y cuál es su sintaxis?

Un **objeto literal** es una estructura de datos de JavaScript que agrupa bajo una misma variable un conjunto de características (llamadas **propiedades**) y comportamientos (llamados **métodos**) correspondientes a una entidad del mundo real (un producto, un alumno, un coche o una cuenta bancaria).

Se denomina *literal* porque se declara directamente entre llaves `{ ... }` formando pares de `clave: valor` separados por comas, sin requerir clases previas ni funciones constructoras.

💡 **TRES FORMAS HABITUALES DE CREAR UN OBJETO LITERAL**

```
// Forma 1: Creación directa con sus propiedades iniciales (la más habitual)
const alumno = {
  nombre: "Elena",
  edad: 20,
  matriculado: true
};

// Forma 2: Creación de objeto vacío y asignación posterior
const ficha = {};
ficha.curso = "Confección Web";
ficha.aula = 104;

// Forma 3: Creación abreviada a partir de variables existentes (ES6)
const marca = "Logitech";
const modelo = "MX Master";
const raton = { marca, modelo }; // Equivale a { marca: marca, modelo: modelo }
```

📋 Glosario Técnico Fundamental

TÉRMINO	DESCRIPCIÓN DIDÁCTICA	EJEMPLO DE CÓDIGO
Llaves `{ }`	Delimitan el inicio y el final de la definición del objeto literal.	`const coche = { ... };`
Miembro	Cualquier elemento interno del objeto (tanto propiedades como métodos).	`marca` y `arrancar()` son miembros
Propiedad	Dato asociado a una clave (String, Number, Boolean, Array u Objeto).	`marca: "Toyota", puertas: 5`
Método	Función perteneciente al objeto que representa una acción o comportamiento.	`arrancar() { return "En marcha"; }`
Punto `obj.prop`	Notación preferida y legible para leer, escribir o actualizar propiedades.	`coche.marca = "Seat";`
Corchetes `obj[clave]`	Acceso mediante cadenas de texto o nombres guardados en variables dinámicas.	`coche["marca"]` o `coche[variable]`

🔐 2. Miembros públicos y privacidad en notación literal

En el paradigma orientado a objetos distinguimos entre **miembros públicos** (accesibles y modificables desde cualquier parte del programa) y **miembros privados** (protegidos en memoria para que solo puedan manipularse desde dentro de la entidad).

REGLA 1: POR DEFECTO	REGLA 2: ¿SINTAXIS #?	REGLA 3: PRIVACIDAD REAL
100% PÚBLICO	SOLO EN CLASS	CLAUSURA / IIFE

> ⚠️ **¿Se puede usar `#privado` en un objeto literal?**
> **NO.** La sintaxis moderna de almohadilla (`#saldo`) de ECMAScript solo es válida dentro de declaraciones `class`. Intentar escribir `const obj = { #dato: 10 };` provocará un `SyntaxError` fatal en el navegador.

A) Convención de Privado con Guion Bajo `_` (Falsa Privacidad)

Históricamente en JavaScript se antepone un guion bajo para advertir a otros programadores de que una propiedad no debería tocarse desde fuera, pero **no existe ninguna protección técnica real**:

```
const usuario = {
  nombre: "Marta",          // Propiedad pública
  _claveSecreta: "1234",    // Convención visual (¡pero sigue siendo modificable!)
  saludar() {
    return `Hola, soy ${this.nombre}`;
  }
};

usuario._claveSecreta = "9999"; // ⚠️ Modificación externa permitida por JavaScript
```

B) Privacidad REAL en Notación Literal mediante Clausura (IIFE)

Para lograr encapsulación estricta en un objeto literal se utiliza el **Patrón Módulo**: una función autoejecutable donde las variables locales quedan retenidas en la clausura léxica y solo se expone el objeto con los métodos autorizados:

```
const cajaFuerte = (function() {
  // Variables privadas: INACCESIBLES desde fuera del módulo
  let _pinSecreto = "9876";
  let _dinero = 1000;

  function _registrar(texto) {
    console.log(`[Auditoría]: ${texto}`);
  }

  // Devolvemos el OBJETO LITERAL con SOLO los métodos públicos
  return {
    consultarSaldo(pin) {
      if (pin !== _pinSecreto) return "PIN incorrecto";
      _registrar("Consulta autorizada");
      return `${_dinero} €`;
    }
  };
})();

console.log(cajaFuerte._dinero);              // Devuelve: undefined (protegido)
console.log(cajaFuerte.consultarSaldo("9876")); // Devuelve: "1000 €"
```

🏷️ 3. Ejemplo 1: Creación, lectura, modificación y borrado (`delete`)

A diferencia de lenguajes de tipado estricto (como Java o C#), los objetos literales en JavaScript son **completamente dinámicos**: permiten añadir propiedades en cualquier instante o eliminar claves existentes mediante el operador nativo `delete`.

```
// 1. Creación de un objeto literal con propiedades de distintos tipos
const producto = {
  codigo: "A-104",
  nombre: "Teclado Mecánico RGB",
  precio: 59.90,
  enStock: true
};

// 2. Lectura de propiedades: notación de punto vs notación de corchetes
console.log(producto.nombre);      // "Teclado Mecánico RGB"
console.log(producto["precio"]);   // 59.9

// 3. Modificación de una propiedad existente
producto.precio = 49.95;

// 4. Incorporación dinámica de una nueva propiedad no definida al inicio
producto.categoria = "Periféricos";

// 5. Eliminación de una propiedad mediante el operador 'delete'
delete producto.codigo;

console.log(producto);
/* Resultado en consola:
{
  nombre: "Teclado Mecánico RGB",
  precio: 49.95,
  enStock: true,
  categoria: "Periféricos"
}
*/
```

🔍 ¿CUÁNDO USAR CORCHETES `OBJ[CLAVE]` ?

Los corchetes son obligatorios cuando el nombre de la propiedad está almacenado en una **variable** o contiene caracteres especiales (como guiones o espacios):

```
const campo = "precio";
console.log(producto[campo]); // 49.95
// producto.campo buscaría literalmente la clave "campo"
```

💡 Comprobar si existe una propiedad con `in`

El operador `in` permite averiguar si una clave pertenece al objeto tras haber aplicado `delete`:

```
if ("codigo" in producto) {
  console.log(producto.codigo);
} else {
  console.log("Código no disponible");
}
```

⚠️ Buenas prácticas con `const` en objetos literales

Declarar un objeto con `const producto = { ... };` **no impide modificar sus propiedades internas**. Lo que `const` impide es reasignar la variable a otro valor (por ejemplo: `producto = "otro";` lanzaría error).

4. Ejemplo 2: Métodos y la palabra reservada `this`

Un **método** es una propiedad cuyo valor es una función. La palabra clave `this` dentro de un método hace referencia directa **al propio objeto** que contiene y ejecuta la acción, permitiéndole consultar y modificar sus propiedades internas sin depender de variables externas.

```
// Objeto que modela una cuenta bancaria con saldo y métodos operativos
const cuentaBancaria = {
  titular: "Lucía Navarro",
  saldo: 200,

  // Método para ingresar dinero al saldo propio
  ingresar(cantidad) {
    this.saldo += cantidad;
    return `Ingreso de ${cantidad}€. Saldo actual: ${this.saldo}€`;
  },

  // Método para retirar dinero con control de saldo suficiente
  retirar(cantidad) {
    if (cantidad > this.saldo) {
      return "Operación denegada: saldo insuficiente.";
    }
    this.saldo -= cantidad;
    return `Retirada de ${cantidad}€. Saldo restante: ${this.saldo}€`;
  }
};

console.log(cuentaBancaria.ingresar(50));  // "Ingreso de 50€. Saldo actual: 250€"
console.log(cuentaBancaria.retirar(100)); // "Retirada de 100€. Saldo restante: 150€"
console.log(cuentaBancaria.retirar(500)); // "Operación denegada: saldo insuficiente."
```

¿POR QUÉ ES IMPRESCINDIBLE USAR `this.saldo` ?

Si escribes simplemente `saldo += cantidad;` omitiendo `this.`, el intérprete de JavaScript buscará una variable global o local llamada `saldo` fuera del objeto y provocará un error crítico de ejecución:

ERROR SIN THIS	FORMA CORRECTA CON THIS
`ReferenceError: saldo is not defined`	`this.saldo += cantidad;`

¡PELIGRO DOCENTE! No uses funciones flecha en métodos con `this`

Las funciones flecha (`() => {}`) **no tienen su propio contexto** `this`, sino que lo heredan del ámbito padre (en navegadores, el objeto global `window`). Declarar `retirar: (cantidad) => { this.saldo -= cantidad; }` fallará porque `window.saldo` es `undefined`. Utiliza siempre la sintaxis de método abreviado: `retirar(cantidad) { ... }`.

🎓 5. Ejemplo 3: Objetos anidados y arrays dentro de objetos

En aplicaciones complejas, las propiedades pueden contener a su vez otros objetos (composición) o listas de datos (arrays), permitiendo accesos encadenados:

```
const estudiante = {
  nombre: "Carlos Ruiz",
  curso: "1º Confección Web",
  direccion: {
    calle: "Av. de los Castros, 42",
    ciudad: "Santander",
    cp: "39005"
  },
  calificaciones: [7.5, 8.0, 9.5],
  calcularMedia() {
    let suma = 0;
    for (const nota of this.calificaciones) {
      suma += nota;
    }
    return (suma / this.calificaciones.length).toFixed(2);
  }
};

console.log(estudiante.direccion.ciudad);      // "Santander" (acceso encadenado con puntos)
console.log(estudiante.calificaciones[0]);     // 7.5 (acceso a array interno)
console.log(estudiante.calcularMedia());       // "8.33"
```

🛍️ 6. Ejemplo 4: Catálogos con Arrays de Objetos

El patrón arquitectónico más habitual en desarrollo web consiste en almacenar una lista donde **cada posición es un objeto literal** con idéntica estructura de propiedades:

```
const catalogo = [
  { id: 1, titulo: "Monitor 24 pulgadas", precio: 140, disponible: true },
  { id: 2, titulo: "Ratón Inalámbrico",   precio: 25,  disponible: false },
  { id: 3, titulo: "Alfombrilla XXL",     precio: 18,  disponible: true }
];

// Recorrido con bucle for...of para procesar solo los disponibles
for (const item of catalogo) {
  if (item.disponible) {
    console.log(`✓ ${item.titulo} - ${item.precio} €`);
  }
}
```

> 💡 **Ventaja frente a arrays paralelos**
> Evita el uso de 3 arrays separados (`titulos[]`, `precios[]`, `disponibles[]`). Con un array de objetos, al ordenar o eliminar un artículo, todos sus datos viajan unidos de forma atómica y sin riesgo de desincronización.

🔤 7. Ejemplo 5: Recorrido de claves y conversión con formato JSON

Para inspeccionar objetos en bucles o enviarlos a servidores o `localStorage`, JavaScript ofrece herramientas estándar integradas:

```
const configuracion = { tema: "oscuro", idioma: "es", notificaciones: true };

// 1. Obtener listas de claves o valores del objeto
console.log(Object.keys(configuracion));    // ["tema", "idioma", "notificaciones"]
console.log(Object.values(configuracion));  // ["oscuro", "es", true]

// 2. Convertir el objeto literal de JS en una cadena de texto JSON
const textoJSON = JSON.stringify(configuracion);
console.log(textoJSON); // '{"tema":"oscuro","idioma":"es","notificaciones":true}'

// 3. Reconstruir un objeto literal vivo a partir del texto JSON
const objetoRecuperado = JSON.parse(textoJSON);
console.log(objetoRecuperado.tema); // "oscuro"
```

🛑 8. Top 5 Errores Frecuentes y Soluciones

ERROR COMÚN	CAUSA DEL FALLO	SOLUCIÓN CORRECTA
`{ nombre = "Ana" }`	Usar el signo `=` en lugar de dos puntos `:` dentro de las llaves del objeto.	Separar siempre clave y valor con dos puntos: `{ nombre: "Ana" }`.
`return saldo;`	Omitir `this.` en un método. El motor busca una variable externa y lanza `ReferenceError`.	Anteponer siempre `this.` para referirse al miembro interno: `this.saldo`.
`saludar: () ⇒ { ... }`	Usar funciones flecha como métodos. El contexto `this` apunta a `window`, no al objeto.	Usar sintaxis abreviada estándar: `saludar() { return this.nombre; }`.
`obj.campoVariable`	Intentar acceder con punto usando el nombre de una variable. Busca literalmente la clave `"campoVariable"`.	Usar notación de corchetes sin comillas: `obj[campoVariable]`.
`{ #privado: 10 }`	Intentar definir campos privados de clase dentro de la notación literal.	Usar clausura con IIFE (Patrón Módulo) o convención con guion bajo `_privado`.

> 🎓 **Consejo didáctico del profesor**
>
> Practica siempre revisando la pestaña **Console** de las DevTools (F12). Un error habitual es escribir una coma de más al final en versiones antiguas de navegadores o confundir un objeto literal con una cadena de texto JSON.